section 3.8: Goto and Labels

pages 65-66

A tremendous amount of impassioned debate surrounds the lowly `goto` statement, which exists in many languages. Some people say that `goto`s are fine; others say that they must **never** be used. You should definitely shy away from `goto`s, but don't be dogmatic about it; some day, you'll find yourself writing some screwy piece of code where trying to avoid a `goto` (by introducing extra tests or Boolean control variables) would only make things worse.

page 66

When you find yourself writing several nested loops in order to search for something, such that you would need to use a `goto` to break out of all of them when you do find what you're looking for, it's often an excellent idea to move the search code out into a separate function. Doing so can make both the ``found'' and ``not found'' cases easier to handle. Here's a slight variation on the example in the middle of page 66, written as a function:

```
/* return i such that a[i] == b[j] for some j, or -1 if none */


int findequal(int a[], int na, int b[], int nb)
{
        int i, j;


        for(i = 0; i < na; i++) {
                for(j = 0; j < nb; j++) {
                        if(a[i] == b[j])
                                return i;
                }
        }


        return -1;
}
```

This function can then be called as

```
i = findequal(a, na, b, nb);
if(i == -1)
        /* didn't find any common element */
else    /* got one */
```

(The only disadvantage here is that it's trickier to return `i` and `j` if we need them both.)

Read sequentially: prev next up top